Geometry Studio

Computer Graphics 3D Project Report

Student ID: *replace with your student ID*

May 27, 2026

Abstract

Geometry Studio is an interactive WebGL2 application for demonstrating core computer graphics concepts in three-dimensional space. The project implements primitive geometry generation, model loading, solid/point/line rendering modes, perspective projection controls, affine transformations, lighting, shadows, texture mapping, animation, scene persistence, and an evaluator-focused tour. The application is built with Three.js, TypeScript, Vite, and a keyframe timeline editor, and is packaged as a static release that can be served locally for grading.

Contents

1 Project Overview

The goal of this project is to build a practical 3D graphics studio rather than a collection of isolated demonstrations. The application allows users to create objects, inspect their rendering modes, transform them interactively, adjust the camera projection, configure lights and shadows, apply textures, import external models, and run animations.

The project is designed for two audiences:

- **Course evaluators**, who need to verify that every assignment requirement is implemented clearly.
- **Students and developers**, who need a maintainable source structure that demonstrates how a real-time graphics application is organized.

2 Technology Stack

Technology	Purpose
Three.js 0.184.0	WebGL2 rendering, geometry, materials, lighting, shadows, controls, loaders, and post-processing.
TypeScript 6.0.3	Static typing and safer long-term source maintenance.
Vite 8.0.14	Development server and static production build.
Lucide	UI icon system.
Playwright	Browser smoke tests for release validation.
animation-timeline-js 2.3.5	Canvas-based visual timeline control for transform keyframes.
pdflatex	English report generation from LaTeX source.

Table 1: Main technologies used in the project.

3 Implemented Requirements

3.1 3D Geometry

The application supports the required primitive shapes:

- Box / Cube
- Sphere
- Cone
- Cylinder
- Wheel / Torus
- Teapot

Additional self-researched shapes are also included:

- Torus Knot

- Tube Curve
- Tetrahedron, Octahedron, Dodecahedron, and Icosahedron
- Parametric Surface
- Extruded Star
- Built-in sample drone model for model-loading demonstrations

Primitive geometry is generated using Three.js buffer geometries and normalized so that objects are centered in the scene and placed naturally on the ground plane. This makes the scene easier to inspect and makes object transformations more predictable.

3.2 Rendering Modes

Each object can be displayed in three required modes:

- **Solid**: rendered as a mesh with material and lighting.
- **Points**: rendered as vertices using `THREE.Points`.
- **Lines**: rendered using `WireframeGeometry` and `LineSegments`.

The line mode deliberately uses a real wireframe geometry instead of a single `Line` object. This avoids incomplete edge rendering on complex geometry and imported models.

3.3 Perspective Projection

The project uses `THREE.PerspectiveCamera`. The user can adjust:

- Camera position on the X, Y, and Z axes
- Field of view
- Near clipping plane
- Far clipping plane

Camera presets are provided for front, top, isometric, and reset views. A camera frustum helper can be enabled to make the projection volume visible during evaluation.

3.4 Affine Transformations

Affine transformations are implemented with `TransformControls`. The selected object can be translated, rotated, or scaled using:

- On-screen transform gizmos
- Keyboard shortcuts: T, R, and S
- Numeric inspector fields for X, Y, and Z values
- World-space and local-space transform modes

Each scene object is wrapped in a `THREE.Group`, which provides a stable transform root even when the object's internal visual representation changes between solid, point, and line modes.

3.5 Lighting and Shadows

The lighting system includes:

- Ambient light for global illumination
- Directional light to simulate sunlight
- Point light
- Spot light

The user can control light color, intensity, position, helpers, shadow state, and a light sweep animation. The renderer uses `PCFSoftShadowMap` and a shadow-receiving ground plane to make shadows visible during grading.

3.6 Texture Mapping

The project includes built-in procedural texture presets:

- Checker
- UV grid
- Grid

Users can also upload bitmap images from their computer. Uploaded textures are loaded through `TextureLoader`, assigned to the selected material, and configured with sRGB color space. Repeat values on the X and Y axes can be adjusted from the inspector.

3.7 Model Loading

The application supports external model import in the following formats:

- GLB / GLTF
- OBJ
- STL

The import pipeline uses `LoadingManager` for progress reporting and clear error messages. Imported objects are normalized, centered, scaled to a reasonable size, placed on the ground plane, and added to the same outliner and transformation workflow as primitive objects. Drag-and-drop import is also supported.

3.8 Animation

The application supports a keyframe-first animation workflow. Per-object preset buttons generate editable timeline keyframes instead of hidden procedural motion. The available presets are:

- Spin
- Orbit
- Bounce
- Pulse

The project also includes a light sweep preset for demos; it is baked into light position keyframes so the resulting motion is visible in the same timeline as object motion.

The keyframe timeline provides a resizable bottom editing dock with a playhead, timecode, duration, FPS, Work In/Out range, snapping, loop control, and tracks for object transforms, object appearance, camera properties, and lights. Users can add, delete, duplicate, copy, paste, drag, scrub, save, and reload keyframes. The `Set TRS` timeline command records the selected object's Position, Rotation, and Scale together at the playhead, so a complete pose can be keyed at t_0, changed, and keyed again at t_1 for pose-to-pose interpolation. When the selected object has two or more Position keyframes, the viewport draws a motion path: a curve for the sampled trajectory and points for the authored key positions. This makes the animation relationship from an initial position at time t_0 to a later position at time t_1 visible directly in the scene, instead of only in the timeline row. The left row labels select object, camera, and light tracks directly, and active tracks can be disabled non-destructively to test motion or appearance changes without deleting keyframes. Focus, Keyed, and All row filters keep dense scenes manageable while preserving access to the active track. Object Position, Rotation, and Scale tracks are expanded into X/Y/Z timeline rows for clearer motion inspection while remaining vector tracks in the saved JSON document. Each timeline row also has a diamond key button, allowing users to key a property directly from its row in the style of motion-graphics editors. Named timeline markers add cue points for animation beats, camera moves, and demo sections. Frame-accurate navigation is available through previous/next frame buttons and keyboard shortcuts, while selected keyframes can be nudged one frame left or right for precise retiming. A compact keyframe editor allows precise numeric edits to keyframe time and vector or scalar values. Linear, Easy Ease, and Hold interpolation are available through direct timeline buttons, synchronized with a compact curve preview and F9-style keyboard shortcuts. The timeline also provides an active-track value graph that samples the same runtime interpolation function used by playback, so Hold, Linear, and Easy Ease segments shown in the graph match the object, camera, light, or material values seen while scrubbing. The graph draws key points on top of the curves and supports horizontal and vertical dragging to retime a keyframe and edit a keyed channel value; each graph drag is recorded as one undoable edit. Graph retiming follows the timeline Snap setting, and Shift-dragging constrains the edit to the dominant drag direction for more precise graph adjustments. The editor stores rotation keyframes in readable Euler degrees and evaluates rotation playback per axis, preserving authored full turns such as 0° to 360°.

4 Advanced Features

4.1 Scene Persistence

The application can save and load a typed JSON scene document. The document stores:

- Scene version and save time
- Object names, types, render modes, materials, colors, textures, and transforms
- Object animation settings
- Keyframe timeline settings, object tracks, and keyframe values

- Camera position, target, field of view, near plane, and far plane
- Display settings such as UI density, grid, axes, stats, and frustum helper
- Lighting configuration

External model binary data is not embedded into the JSON document. If a saved scene references an imported model, the loader restores a built-in sample placeholder so the scene workflow remains usable.

4.2 Undo and Redo

Undo and redo are implemented through a command-history system that stores snapshots of the scene document. This allows the project to restore prior states after changes such as object creation, deletion, material changes, render mode changes, camera changes, lighting changes, and timeline keyframe edits.

4.3 Keyframe Timeline Runtime

The keyframe timeline is implemented as a separate editor layer and runtime layer. The UI layer uses `animation-timeline-js` for visual keyframe rows, selection, dragging, snapping, and playhead interaction. Geometry Studio owns the actual scene timeline schema so saved JSON files remain stable and independent from the UI library.

At runtime, transform and property tracks are evaluated directly from the saved timeline document:

- Position and Scale write vector values to the selected object.
- Rotation stores readable Euler degrees and converts them to radians at playback time, preserving full-turn animations.
- Hold, Linear, and Easy Ease interpolation are evaluated per keyframe segment, so mixed timing within a single track is supported.

If an object has active transform timeline tracks, timeline playback overrides the older preset transform animation for that same object. This avoids ambiguous motion and makes the evaluator-visible behavior deterministic.

4.4 Evaluation Tour

The Evaluation Tour is designed specifically for grading. It creates a scene that demonstrates:

- Required primitive geometry
- Solid, point, and line rendering
- Perspective camera and frustum helper
- Affine transform workflow
- Ambient, spot, and shadowed lighting
- Texture mapping
- Model-loading workflow through the built-in sample model

- Multiple animation types

Toast messages appear in sequence to explain which requirement is being shown.

4.5 Cinematic Demo and Screenshot Export

The Cinematic Demo creates a visually stronger scene with different geometry, materials, render modes, textures, and animations. The screenshot button exports the current WebGL canvas as a PNG image.

4.6 Telemetry

The telemetry panel displays FPS, draw calls, triangles, lines, points, geometry count, texture count, and object count. These values are based on `renderer.info` and help demonstrate rendering cost during manual evaluation.

5 Captured Screenshots

The following screenshots were captured from the static `Release/` build served locally through `python3 -m http.server 8080`. They demonstrate the final evaluator-facing interface and the major graphics features.

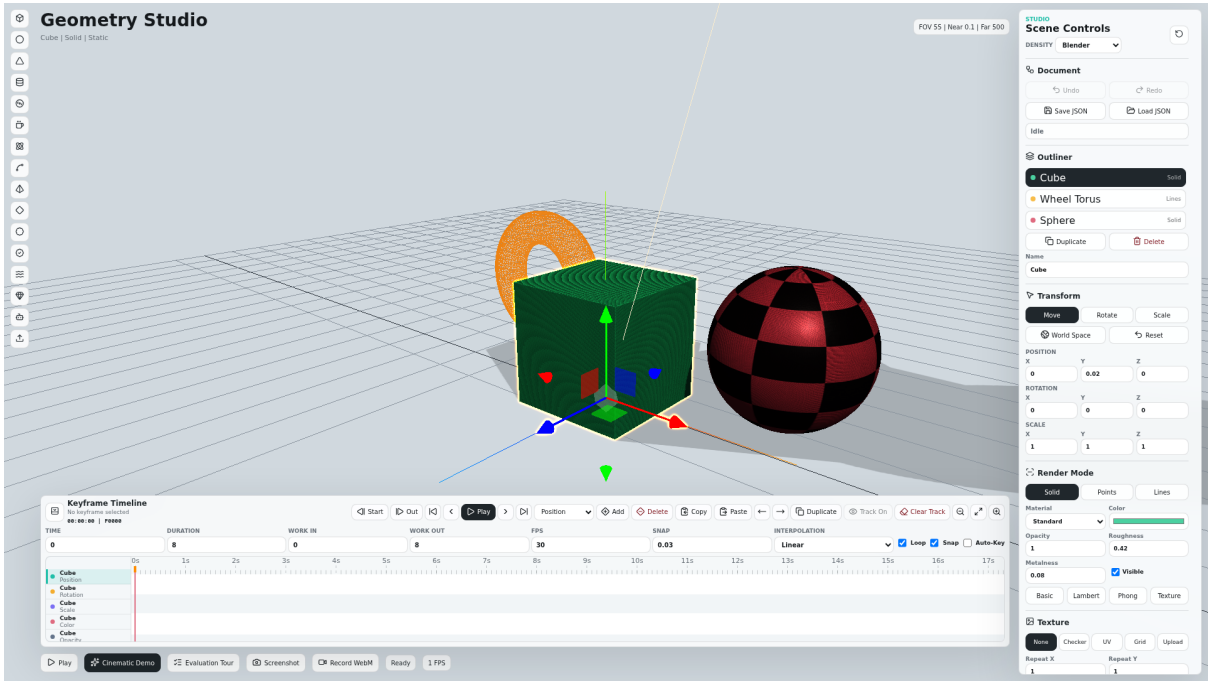

Figure 1: Default Geometry Studio scene with primitive geometry, render modes, inspector controls, transform gizmo, grid, and timeline dock.

6 Source Architecture

The upgraded source code is split into focused modules:

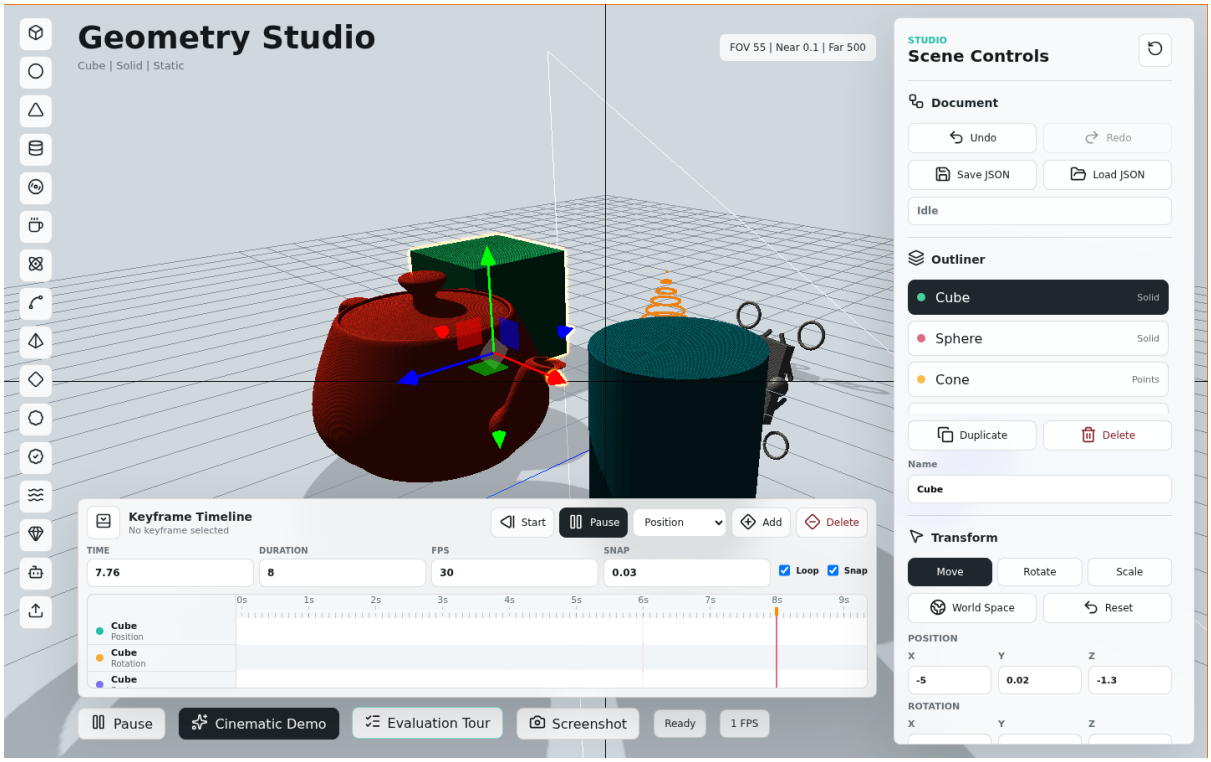

Figure 2: Evaluation Tour scene showing required primitives, shadows, camera frustum helper, lighting helpers, and multiple rendering modes.

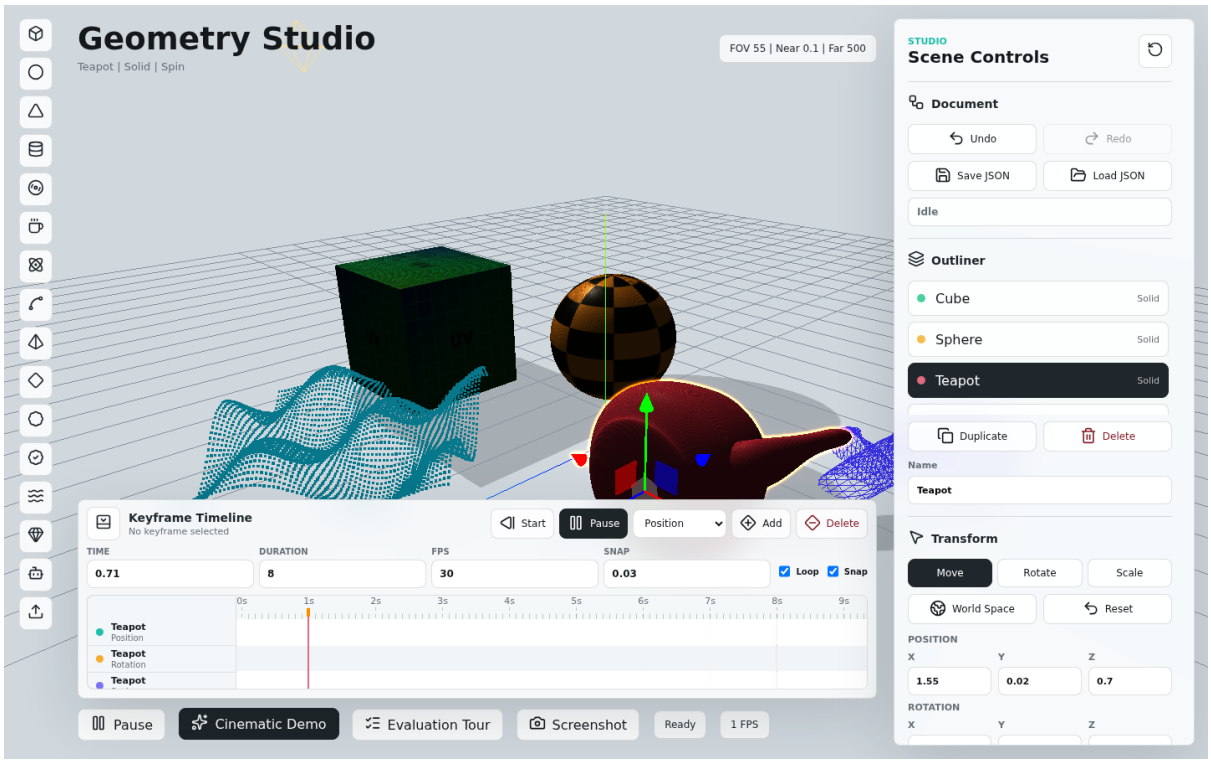

Figure 3: Cinematic rendering demo with mixed primitives, textures, point/line rendering, lighting, shadows, and animation playback.

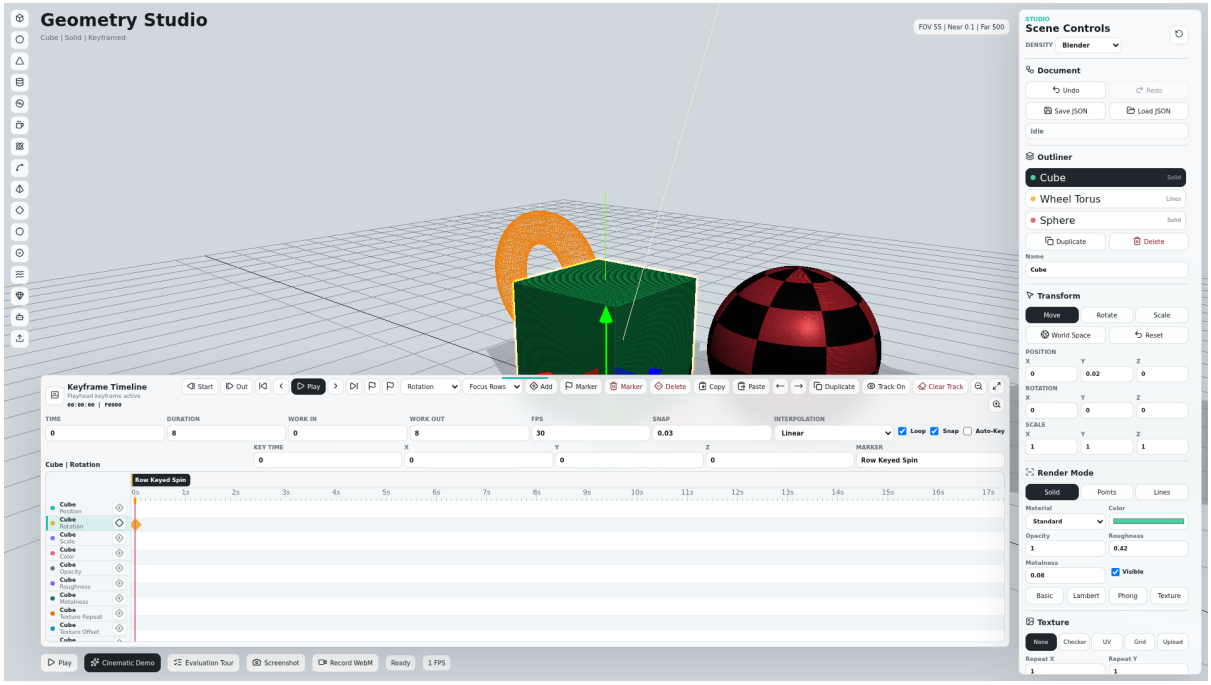

Figure 4: Resizable keyframe timeline showing editable track rows, markers, and selected keyframe values in the timeline workflow.

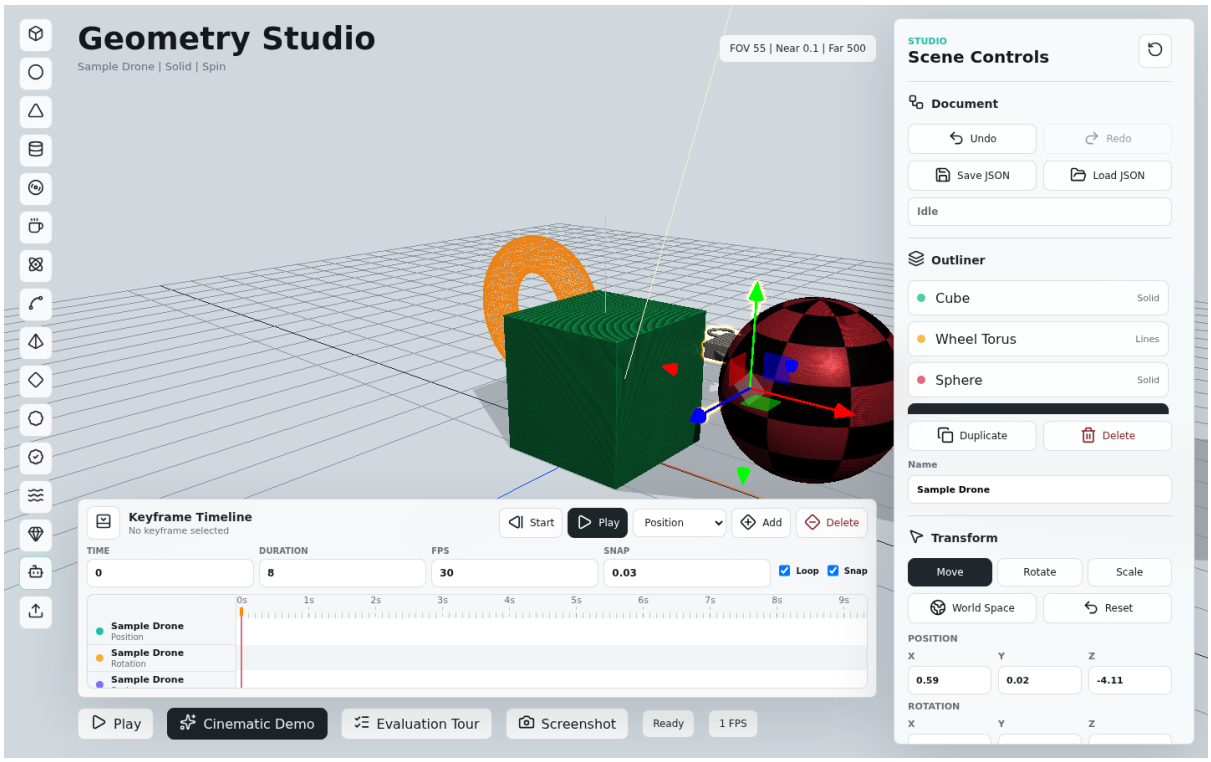

Figure 5: Built-in sample model workflow demonstrating model-style objects in the same outliner, transform, render, and animation system.

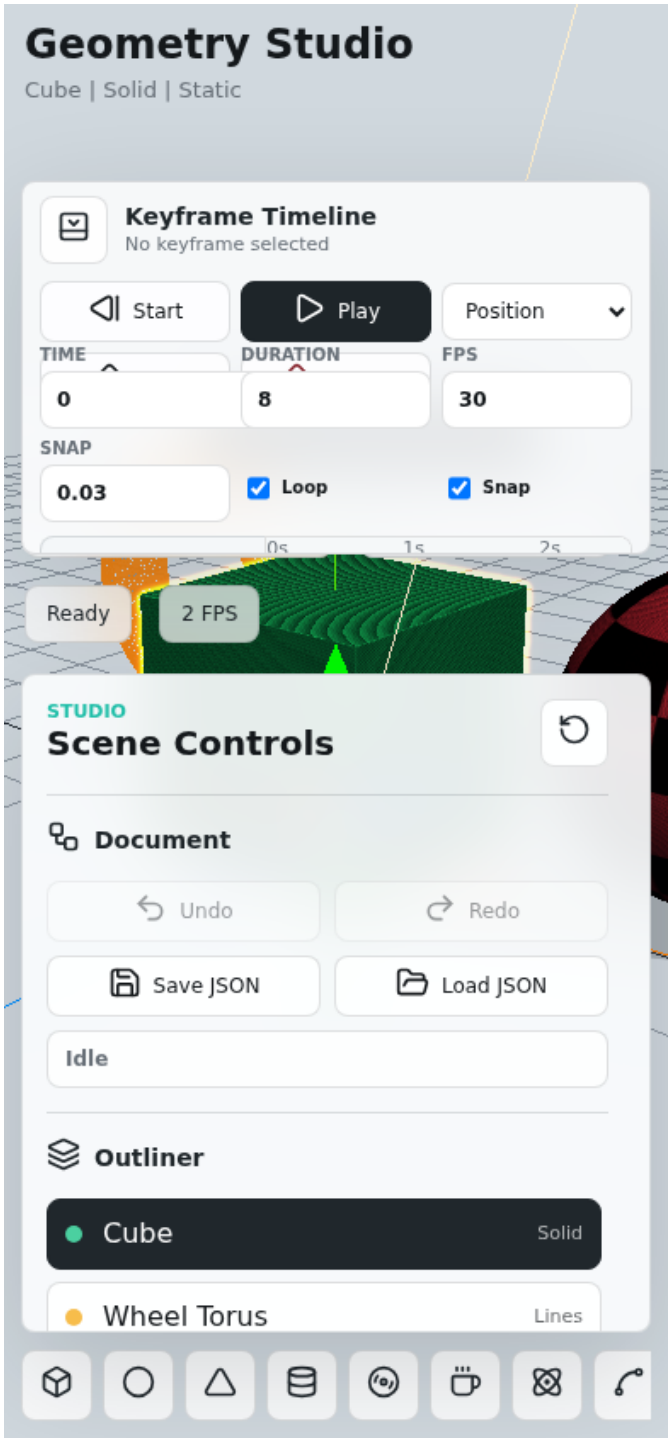

Figure 6: Responsive mobile layout with timeline, inspector, viewport, and horizontal tool rail.

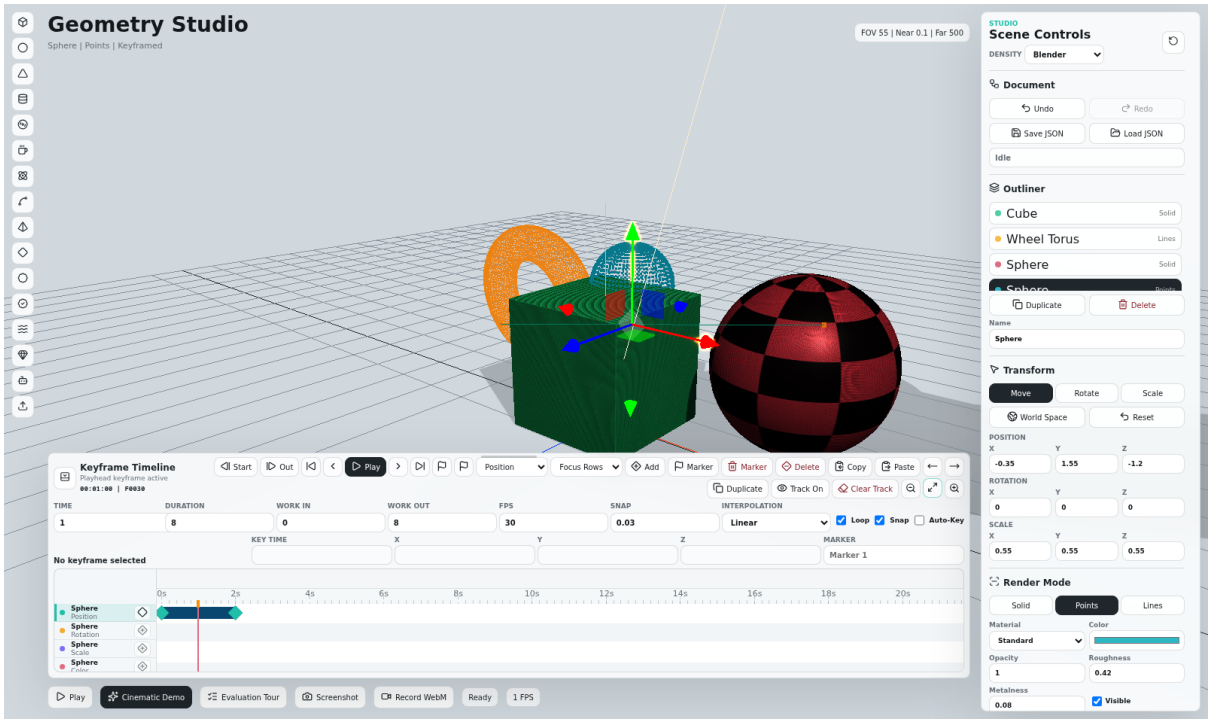

Figure 7: Position keyframes with the selected object's viewport motion path, showing the spatial interpolation from one keyed position to the next.

7 Rendering and Performance Considerations

The renderer is configured for WebGL2 with ACES tone mapping, sRGB output color space, soft shadows, and post-processing outline selection. Resize handling uses the canvas client size and caps the drawing buffer pixel count to avoid excessive GPU load on high-DPI displays.

The project explicitly disposes geometries, materials, textures, and object URLs when objects are rebuilt or removed. This follows standard Three.js resource management practices and reduces the risk of memory leaks during long editing sessions.

8 Planned Improvements

The current release focuses on a stable real-time WebGL editor. Future work should improve rendering quality while preserving interactivity and grading clarity.

1. **Rendering Lab**: add tone mapping, exposure, shadow quality, PBR material presets, environment lighting controls, and a clearer rasterization versus ray tracing explanation.
2. **Environment Lighting**: use local environment assets and `PMREMGenerator` to support physically-based reflections and roughness-dependent lighting.
3. **OBJ + MTL Import**: extend OBJ import so users can load companion material files and texture images for models such as teacups, furniture, or scanned objects.
4. **Post-Processing**: add controlled toggles for bloom, ambient occlusion, color grading, and anti-aliasing passes.

Module	Responsibility
editor/types.ts	Shared application types such as SceneEntry, LightRig, and SceneDocument.
editor/documents.ts	Scene JSON serialization and validation.
editor/commands.ts	Undo/redo snapshot history.
animation/timelineSchema.ts	Timeline defaults, validation, migration, snapping, and object-track helpers.
animation/timelineEditing.ts	Pure keyframe editing operations such as source resolution, copy, paste, duplicate, frame nudge, and numeric keyframe edits.
animation/interpolation.ts	Shared per-keyframe evaluator for Hold, Linear, and Easy Ease timing.
animation/timelinePlayer.ts	Applies object Position, Rotation, and Scale tracks directly to scene objects.
scene/primitives.ts	Primitive geometry, extra shapes, labels, and built-in sample model.
scene/materials.ts	Materials, texture presets, and solid/point/line visual construction.
scene/lights.ts	Stage, light rig, shadows, helpers, and light sweep.
scene/importers.ts	GLB, GLTF, OBJ, and STL import pipeline.
renderer/pipeline.ts	WebGLRenderer, EffectComposer, OutlinePass, and resize handling.
animation/timeline.ts	Legacy procedural animation helper retained for compatibility.
ui/template.ts	DOM template for the editor UI.
ui/density.ts	UI density loading, persistence, and root layout mode application.
ui/timelinePanel.ts	Adapter between the timeline UI component and editor commands.
utils/resourceTracker.ts	Explicit disposal of geometries, materials, textures, and URLs.

Table 2: High-level source structure.

5. **Timeline Polish**: add auto-key, duplicate keyframes, camera tracks, and light/material tracks.
6. **Path-Traced Preview**: evaluate a separate progressive still preview using a GPU path-tracing library. This should remain optional because the main editor is designed for real-time rasterized interaction.

9 Testing

The project includes automated browser tests with Playwright:

- Startup rendering and non-empty canvas verification
- Toolbar and inspector visibility

- Object creation
- Render mode switching
- Responsive mobile layout smoke test
- Undo and redo
- Scene JSON loading
- Keyframe timeline creation and JSON save verification
- Evaluation Tour activation

The main verification commands are:

```
npm run typecheck
npm run build
npm run test:smoke
```

10 How to Run

To run from source:

```
cd Geometry_Studio/Source
npm install
npm run dev
```

To run the static release:

```
cd Geometry_Studio/Release
python3 -m http.server 8080
```

Then open:

```
http://localhost:8080
```

11 Conclusion

Geometry Studio satisfies the project requirements for 3D geometry, rendering modes, perspective projection, affine transformations, lighting, shadows, texture mapping, model loading, and animation. The upgraded version also adds scene persistence, undo/redo, import progress, drag-and-drop, telemetry, an Evaluation Tour, screenshot export, and a modular source architecture. These features make the project easier to evaluate, easier to maintain, and more impressive as a complete computer graphics demonstration.